

# Lab 3: Advanced SQL 1

In this lab you will explore more advanced SQL queries, including joins, where you access data from more than one table using a join constraint, some more advanced types of joins, aggregate queries, where you apply an aggregation function, such as sum or average, to the results, and explore grouping through the use of the GROUP BY and HAVING clauses.

## Objectives:

- Write queries that join two or more tables in a query.
- Apply the correct join constraint to implement a query specification.
- Get started with more complex joins
- Write queries that aggregate data in groups
- Apply constraints to groupings with a HAVING clause.

There are two main types of joins: *inner joins*, where only rows matching the *join constraint* are included in the result set, and *outer joins*, where all of the rows of one of the tables is included in the result set regardless of if it matches the join constraint.

## Preparation:

You will be working with the ***project\_sales*** database. Create a database called `project_sales` in postgres, connect to the `project_sales` database with the `\c` command in psql, then execute the build script in the file [project\\_sales.sql](#).

**Comments on developing more complex queries:** As with writing code in a programming language, it helps to develop your queries incrementally.

***Use psql to develop your queries***, then submit to gradescope when you are satisfied you have a correct result set. Why?

1. By using psql, you will develop query writing skills by working with a real database system, where you have to look at the database structure and data when developing your queries. Gradescope does not provide any views of the database structure and data stored in tables. You will not have Gradescope around in a non-academic setting.

2. The psql shell will provide error messages which contain information about the error. This makes developing queries more efficient because you get clues about the type of error and what the system thinks is a possible problem. Gradescope is only an autograder that provides very limited feedback. It will take you a lot longer to troubleshoot your queries by using Gradescope to develop your queries.

The database used in this lab is simple, so it is easy to check your results, but this is more difficult in real-world databases. Try executing a join first and check that it looks correct before adding more constraints.

You can execute smaller queries to explore parts of the database that your final query will use. You can also execute a “select \* etc.” query to see all of the columns in a join and then you will have a better idea of exactly what columns you need in your final select clause. You can start with a simpler query and then add constraints as you check and develop the final query.

Use the LIMIT keyword if necessary to limit the size of the result when developing/troubleshooting.

Another technique is to execute a probing query with one specific “hard-coded” constraint, such as a specific name that is in the data set you are looking for. If you get the result you are looking for for that specific name, then you can generalize the query.

Remember that every join should have a join constraint. Accessing data from two tables without a join constraint will produce the cross product of the tables; that means each row of table A will be matched with each row of table B. The result set will contain a large number of rows: rows of A \* rows of B. This is never a useful result.

**For the best learning experience:** run your query against the database on your machine while developing it. Do not use gradescope to develop and debug your query until you are sure it is correct. Explore the database and use the techniques above to check that your query is correct. That way you will (1) get to “know” the database, and (2) develop good querying skills for when you are in an environment where there is no gradescope evaluator.

Let us know if you have tried to get a query to work but got stuck or have any questions!

## Part 1: Join Queries

The queries in this part require data from more than one table. When joining two tables, you will need to use the correct *join constraint* (which columns from each table to join on) and apply any other constraints required by the query specification to get the correct result sets. Typically, you are joining a foreign key in one table to a primary key in another table.

Note that you can accomplish the join by using the appropriate JOIN operator or specify the join constraint columns in a WHERE clause explicitly.

There are 2 types of joins: inner and outer joins.

For documentation on postgres JOIN operators:

<https://www.postgresql.org/docs/current/tutorial-join.html>

Check your results in postgres to verify you have the correct query results.

Write the following SQL queries.

**Q1:** Write a query that retrieves a list of product names, prices, product categories, manufacturer, the month, and the number of items sold in that month (represented by the attribute `sold` in `sales`). Sort alphabetically by product name. You should have 13 rows. Submit the query in file Q1.sql.

*– Did you get an error about an “ambiguous” column name? If so, remember that the full name of a column is of the form: `tablename.columnname`. When you join two tables you are creating a single table, so if there are duplicate column names you have to specify exactly which column you want by including the table name, or the alias if you are using an alias for table names.*

*Sometimes it can help to run a query with `SELECT *` and see what columns are returned. It may make it easier to see which columns you want to specify in the final query.*

**Q2:** Write a query that returns a list of products, price, category, manufacturer, and country, sorted alphabetically by country; ensure that there are no null values in the output. Remember that you can develop the query incrementally. Try the join to see what columns have null values and then augment your query to exclude them. You should have 6 rows in the final result set. Submit the query in file Q2.sql.

**Q3:** Generate a sales report for the month of June. The report should include the product name, price, category, manufacturer, country, and the amount sold. Think about the tables you need to access for the data needed in this report and how to join them. Your final output will have 2 rows on the provided dataset. Submit the query in file Q3.sql.

**Q4:** During which months of the year have companies based in Japan sold products? Your query should return a single column which is a list of the appropriate months. You want to exclude duplicates in this report. You should have 5 rows. Submit the query in file Q4.sql. Hint: what tables do you need to access to generate this report? Look at the primary and foreign key definitions to see how to connect these tables.

**Q5:** Write a query to list all employee names and the projects that each is involved in. Your result has two columns: name and project. Do not omit any employees even if they don't have a project. Note that an employee will be listed more than once if they are linked to more than one project. Hint: Consider the join that you need for this query. Submit the query in file Q5.sql.

**Q6:** Write a query to list all employee names in the first column and the names of their managers in the second column. Employees who don't have a manager should be listed in the output, with NULL in the second column. Hint: You will need a self-join in this query. Submit the query in file Q6.sql.

## Part 2: Aggregation Queries

Aggregation queries involve grouping data and applying aggregate functions to the groups. The following queries involve only one table so you don't need to perform joins. Do not apply any rounding to numeric results in these queries.

Write the following queries:

**Q7:** Retrieve the average price of all products. You do not need to name your average price column with an alias. Your query should return a single value. Submit the query in file Q7.sql.

**Q8:** Retrieve the category and average price for each category of products. Your query should return a list of categories and the average price of all products within each category. You do not need to name your average price column with an alias. Submit the query in file Q8.sql.

**Q9:** Return the category and compute the average price of all products with a price less than \$150, for each category of products. You do not need to name your average price column with an alias. Submit the query in file Q9.sql.

**Q10:** Retrieve the category and average price for each category of products such that only categories with an average price strictly greater than 100.00 are included in the result. Do not name your average price column with an alias. Submit the query in file Q10.sql.

**Q11:** Return the month, and the average number of items sold each month as avg\_sold. Name your average sold column with the column alias avg\_sold. You should have 6 rows. Submit the query in file Q11.sql.

**Q12:** Compute the total number of sold items for those products that have sold more than 3 items in total. Include the product name and the total number of sold items. Use the column alias `items_sold` in your result set. Your output should have 3 rows. Submit the query in file `Q12.sql`.

**Q13:** Return the month and the average number of items sold each month, but only show months that have never had more than 2 sales for an individual product in a single month. You do not need to name your average sold column with an alias. You should have 4 rows. Submit the query in file `Q13.sql`.

**Q14:** Return the category and the average price of products in each category. Only consider products with a price cheaper than \$150, and only include categories where the maximum price of any product in this category is greater than \$80. You do not need to name your average price column with an alias. Submit the query in file `Q14.sql`.

***Check your understanding: think about how you would answer these questions.***

*Check your understanding of the HAVING clause. Can you explain which conditions should go to the HAVING clause vs. which conditions should go to the WHERE clause? Consider queries with a HAVING clause. Can the condition(s) of the HAVING clause move to the WHERE clause? Can a WHERE clause condition move to the HAVING clause?*

**Q15:** For each employee, list the employee's name and how many projects they are involved in. Check that you are reporting the number of projects for all employees and that the counts are correct. Note that some counts could be zero. Submit the query in file `Q15.sql`.

**Q16:** For each product, list how many items it has sold in total. Check that you are showing all products. Order by the total sales in descending order. Hint: To get the correct total to show up for products with no sales, look up the function [COALESCE](#). Submit your query in file `Q16.sql`.

## If you finish early:

Get additional practice through fun SQL-based games.

Have you solved the SQL murder mystery yet? <https://mystery.knightlab.com/>

Also try out Lost at SQL: <https://lost-at-sql.therobinlord.com/>